PlanForge

A Constraint Satisfaction Framework for Apartment Floor-Plan Generation

Designed by Mohammad Saleh Mahdinejad
Artificial Intelligence Fundamentals and Applications

July 28, 2026

Abstract

PlanForge is an educational project that turns apartment floor-plan generation into a concrete constraint satisfaction problem. Students receive a visual framework, a set of JSON problem instances, a public self-check, and a small set of starter files. Their task is to implement the search intelligence: recursive backtracking, consistency checks, variable and value ordering, optional inference, and soft optimization. The project is designed to make CSP concepts visible: each room placement becomes a domain value, each architectural rule becomes a constraint, and the visual solver can replay the actual search path as a tree.

Contents

1 Project Motivation

Classic CSP assignments often use abstract examples such as map coloring or n-queens. Those examples are useful, but they can feel disconnected from the way constraints appear in real design problems. PlanForge uses apartment planning as the domain: rooms must fit inside a fixed shell, must not overlap, must satisfy adjacency and boundary requirements, and should also produce a layout that looks reasonable.

The project has two goals. First, it gives students a strict CSP core where correctness can be graded through hard constraints. Second, it gives advanced students room to improve search quality through inference and soft scoring. This creates a clean separation between required correctness and optional optimization.

2 High-Level Workflow

The framework separates problem definition, domain generation, student search, validation, and visualization. This separation keeps the assignment fair: students implement the algorithmic core, while the framework provides consistent loading, checking, rendering, and reporting.

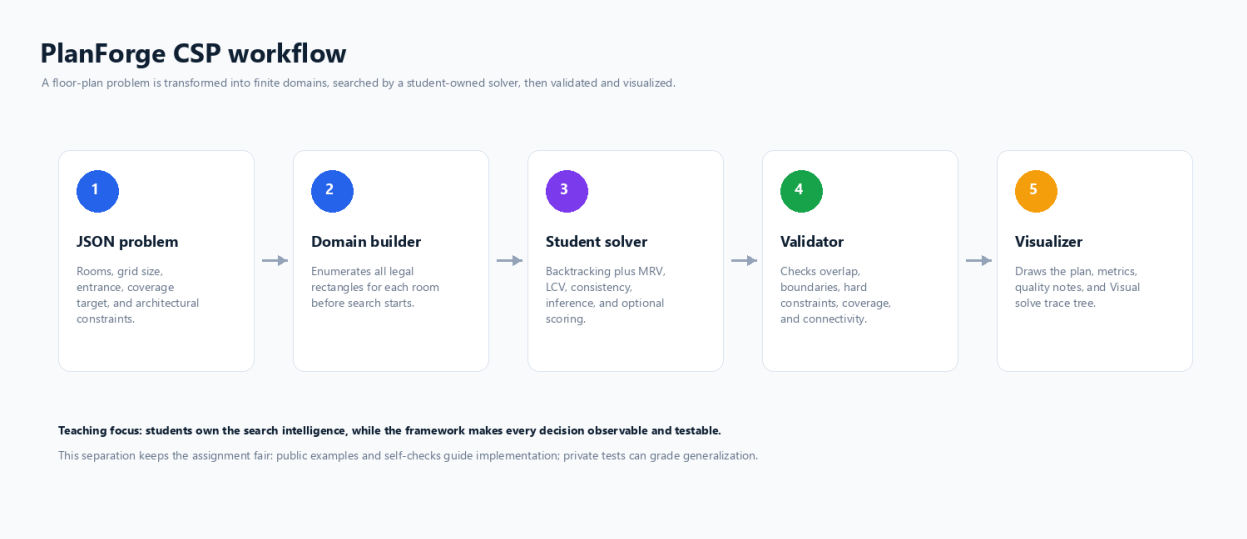

Figure 1: PlanForge transforms a declarative floor-plan case into domains, runs the student solver, validates the result, and visualizes the outcome.

3 CSP Formulation

PlanForge models each apartment case as a finite-domain CSP:

$$\text{CSP} = (X, D, C)$$

where:

- X is the set of variables, one variable per room.
- D_i is the finite domain of legal rectangles for room i.
- C is the set of hard constraints that define validity.

Each domain value is a rectangle:

$$r = (x, y, w, h)$$

where x and y are grid coordinates and w and h are integer dimensions. The domain builder enumerates every rectangle that fits the room's area, width, height, aspect-ratio, and boundary requirements.

3.1 Variables

The variables are room names such as `Hall`, `Bathroom`, `Living`, `Kitchen`, `Bedroom1`, `Storage`, or `Balcony`. A complete assignment maps every room name to exactly one rectangle.

3.2 Domains

A room domain is not the entire grid. It is filtered before search begins. The framework removes rectangles that violate room-level requirements such as:

- minimum and maximum area,
- minimum and maximum width,
- minimum and maximum height,
- maximum aspect ratio,
- exterior access when required.

This precomputation makes the search problem finite, explicit, and inspectable.

3.3 Hard Constraints

The validator and student consistency function work with constraints such as:

Constraint	Meaning
`adjacent(a,b)`	Rooms must share a wall segment.
`not_adjacent(a,b)`	Rooms must not share a wall segment.
`near_entrance(room)`	Room must be close to the entrance point.
`far_from_entrance(room)`	Room must be sufficiently far from the entrance point.
`touches_boundary(room)`	Room must touch the exterior boundary.
`touches_wall(room, wall)`	Room must touch a specific shell wall.
`larger_than(a,b)`	Room `a` must have greater area than room `b`.

Table 1: Representative PlanForge hard constraints.

In addition to explicit JSON constraints, every valid solution must satisfy global requirements: rectangles must stay inside the shell, rooms must not overlap, coverage must exceed the required threshold, and the final room adjacency graph must be connected when connectivity is required.

4 JSON Problem Format

Each example is stored under `planforge/examples/`. The JSON schema is intentionally simple enough for students to read while still expressing meaningful design cases.

Listing 1: Simplified example of a PlanForge problem.

```
{
  "width": 10,
  "height": 8,
  "entrance": [0, 7],
  "minCoverageRatio": 0.72,
  "requireConnected": true,
  "rooms": [
    {
      "name": "Living",
      "minArea": 16,
      "maxArea": 20,
      "minW": 4,
      "maxW": 5,
      "minH": 4,
      "maxH": 4,
      "needsExterior": true
    }
  ],
  "constraints": [
    {"type": "adjacent", "a": "Living", "b": "Kitchen"},
    {"type": "near_entrance", "room": "Hall", "max_distance": 3}
  ]
}
```

The visible examples cover easy, medium, hard, unsatisfiable, and bonus challenge scenarios. The unsatisfiable case is important because it prevents students from assuming that every search must return a layout.

5 Framework Architecture

The public repository is organized around a small framework and a student-owned work area.

Path	Responsibility
`planforge/core/models.py`	Dataclasses for rectangles, room specs, constraints, and CSP problems.
`planforge/core/domain_builder.py`	Enumerates finite rectangle domains for each room.
`planforge/core/geometry.py`	Geometry utilities: overlap, adjacency, distance, connectivity.
`planforge/core/constraints.py`	Parses and evaluates hard constraints.
`planforge/core/loader.py`	Loads JSON cases into `CSPProblem` objects.
`planforge/core/validator.py`	Validates complete assignments.
`planforge/core/quality.py`	Computes architectural soft quality scores.
`planforge/core/engine.py`	Runs the student solver and records instrumentation.
`planforge/core/visualizer.py`	Tkinter studio, layout canvas, metrics, JSON viewer, Visual solve.
`student/`	Student TODO files.

Table 2: Main repository modules and their roles.

6 Student Implementation Surface

Students are expected to edit only the `student/` package. The starter files expose the algorithmic hooks without hiding the important CSP responsibilities.

6.1 Required Files

- `student/solver.py`: implement `solve`, `backtrack`, and `is_complete`.
- `student/consistency.py`: implement partial-assignment consistency checking.
- `student/heuristics.py`: implement MRV and LCV.

6.2 Optional Files

- `student/inference.py`: implement forward checking and AC-3.
- `student/scoring.py`: implement a soft layout score used to choose the best valid plan.

The framework grades only the student-owned interface. This makes it possible to test submissions inside a clean scaffold and prevents students from gaining credit by modifying the visualizer, examples, or public grader.

7 Backtracking and Instrumentation

The student solver receives a `SolverContext`. This object records search statistics and provides hooks that make the algorithm visible.

Listing 2: Typical instrumentation calls inside recursive backtracking.

```
ctx.on_node()
variable = select_unassigned_variable(problem, assignment, domains)
ctx.on_select_variable(variable, assignment)

for value in order_domain_values(problem, variable, assignment, domains):
    ctx.on_assignment_tried()
    ctx.on_consistency_check()
    if is_consistent(problem, assignment, variable, value):
        assignment[variable] = value
        ctx.on_assign(variable, value, assignment)
        ...
        del assignment[variable]
        ctx.on_unassign(variable, assignment)

ctx.on_backtrack()
```

These calls do not solve the CSP for the student. They only expose the student's own decisions to the framework. This is useful for grading, debugging, and the Visual solve mode.

8 Visual Solve Mode

PlanForge includes a graphical mode that replays the search trace. When tracing is enabled, the UI displays the current apartment assignment and a DFS tree representing the explored search path. Current branches, solutions, pruned domains, and backtracked nodes use different visual encodings.

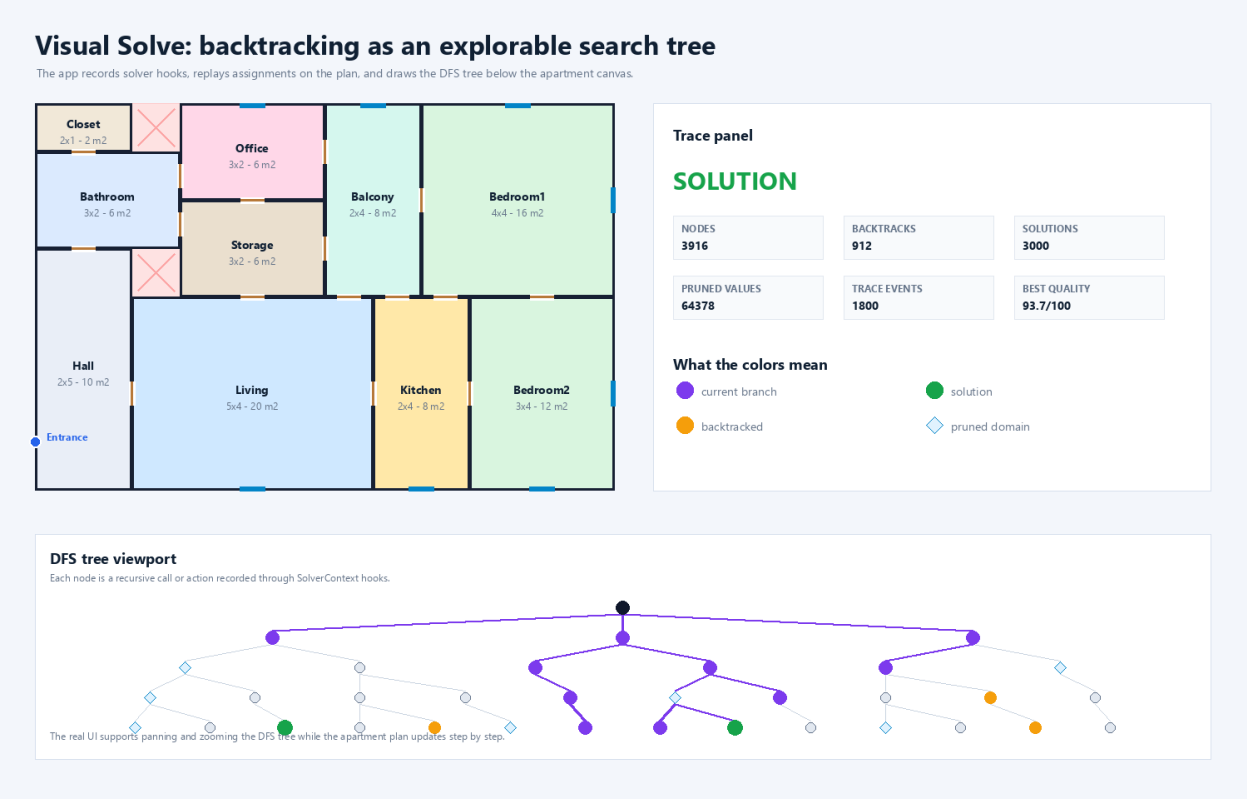

Figure 2: Visual solve mode shows the generated apartment and a DFS search tree driven by `SolverContext` trace hooks.

This mode is valuable pedagogically because it turns backtracking from a hidden recursive process into an observable sequence. Students can see how MRV changes variable selection, how LCV changes branch order, how inference prunes candidate values, and why a branch fails.

9 Layout Visualization

The main studio view displays the selected case, solver status, metrics, and the generated floor plan. Rooms are rendered as colored rectangles. Doors are drawn between adjacent rooms, windows are shown on exterior-facing rooms, and unused cells are highlighted.

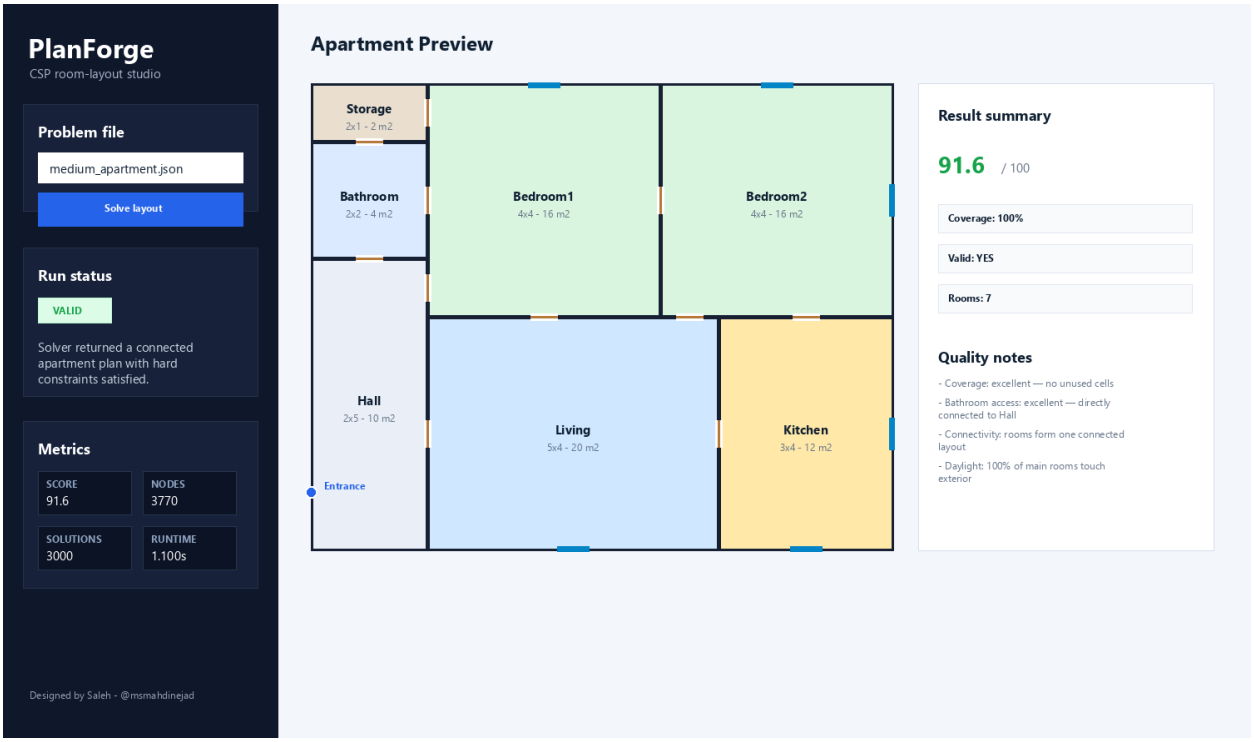

Figure 3: The desktop studio view for a solved medium apartment case.

The visualization intentionally includes both correctness and quality signals. A layout can be valid while still having a weak quality score, which helps students understand the difference between hard constraints and soft objectives.

10 Quality Scoring

Validity and quality are separate. A valid solution satisfies all hard constraints. A high-quality solution also reflects architectural preferences. The framework-owned quality score considers:

- coverage and unused area,
- public circulation from entrance to hall and living room,
- bathroom accessibility,
- connected room graph,
- public/private zoning,
- daylight and exterior access,
- room shape compactness,
- preferred adjacencies such as living-kitchen and kitchen-balcony.

Students can implement `score_assignment` to guide selection among multiple valid assignments. This creates a natural advanced path: after finding any solution, students can search for better solutions.

11 Public Self-Check

The public grader is included as a visible feedback tool. It checks importability, basic heuristic behavior, public example solutions, unsatisfiable detection, instrumentation, and optional performance behavior. It is explicitly not the final grader.

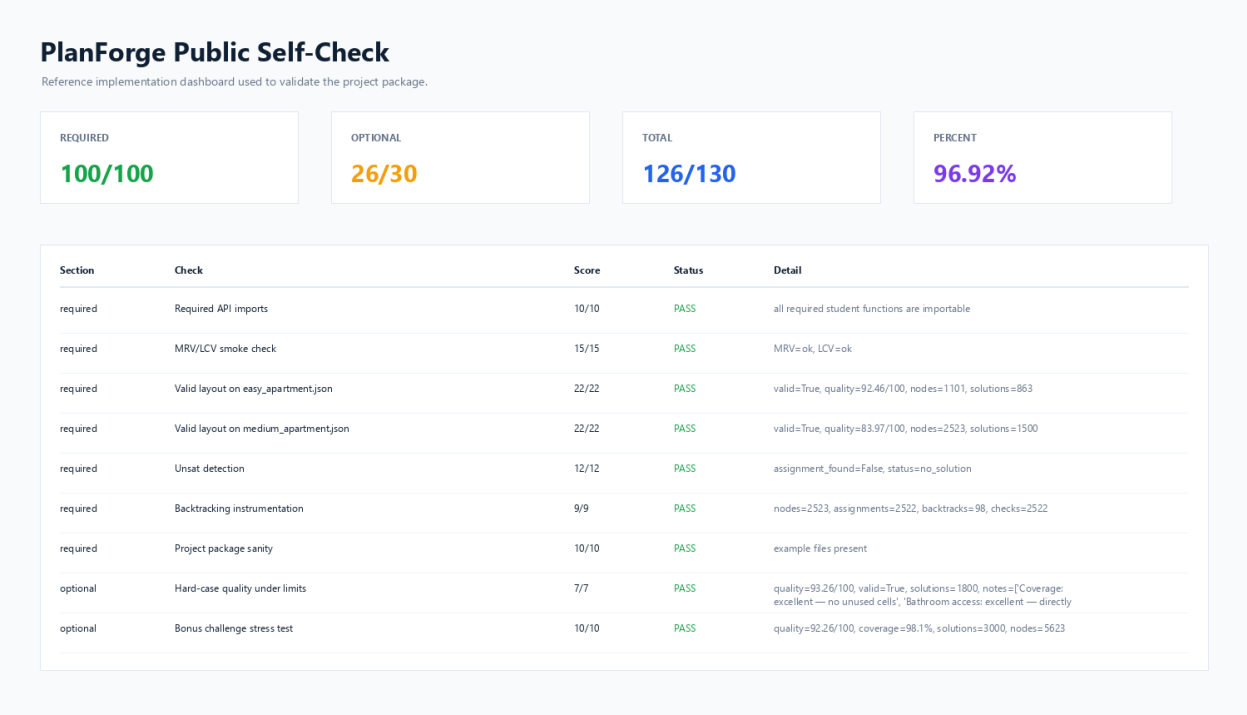

Figure 4: Public self-check dashboard generated from the reference implementation.

The final grading policy should use private cases and a private grader. This protects the assignment from overfitting to visible examples while still giving students enough public feedback to debug their implementations.

12 Required and Optional Grading

The assignment is structured as 100 required points and 30 optional points.

Section	Focus	Examples
Required	Correct CSP search	Backtracking, consistency, MRV, LCV, valid public layouts.
Required	Robustness	Unsatisfiable-case detection and instrumentation.
Optional	Inference	Forward checking, AC-3, stronger pruning.
Optional	Optimization	Multi-solution search and soft layout scoring.

Table 3: Grading categories in PlanForge.

13 Recommended Student Strategy

A practical implementation path is:

1. Implement `is_complete`.
2. Implement a simple recursive backtracking solver using raw domain order.
3. Implement `is_consistent` for boundaries, area, overlap, and checkable constraints.
4. Add `ctx` instrumentation so the public grader and visualizer can observe the search.
5. Add MRV and LCV.
6. Add forward checking.
7. Add AC-3 if time allows.
8. Implement `score_assignment` and keep searching for better valid layouts.

This sequence gives students early visible progress while leaving enough depth for advanced work.

14 Public Repository Policy

The public repository should contain the starter package, visualizer, public examples, public self-check, screenshots, and documentation. It should not contain private grader logic, hidden tests, private grade reports, batch grading outputs, or reference solution source code.

This is why the repository is designed as a clean public release rather than a full instructor archive. The public materials teach and guide; the private materials grade and audit.

15 Conclusion

PlanForge connects CSP theory to a concrete design problem. It preserves the core ideas of AI search while giving students visual feedback and a meaningful optimization target. The result is an assignment where algorithmic choices are visible: variable ordering changes the tree, inference shrinks the search space, and soft scoring improves the final layout.